Dominando Docker y Docker Compose para Python

De scripts aislados a arquitecturas completas con bases de datos en tu entorno local.



Gemini Notebook

El eterno problema de las dependencias locales

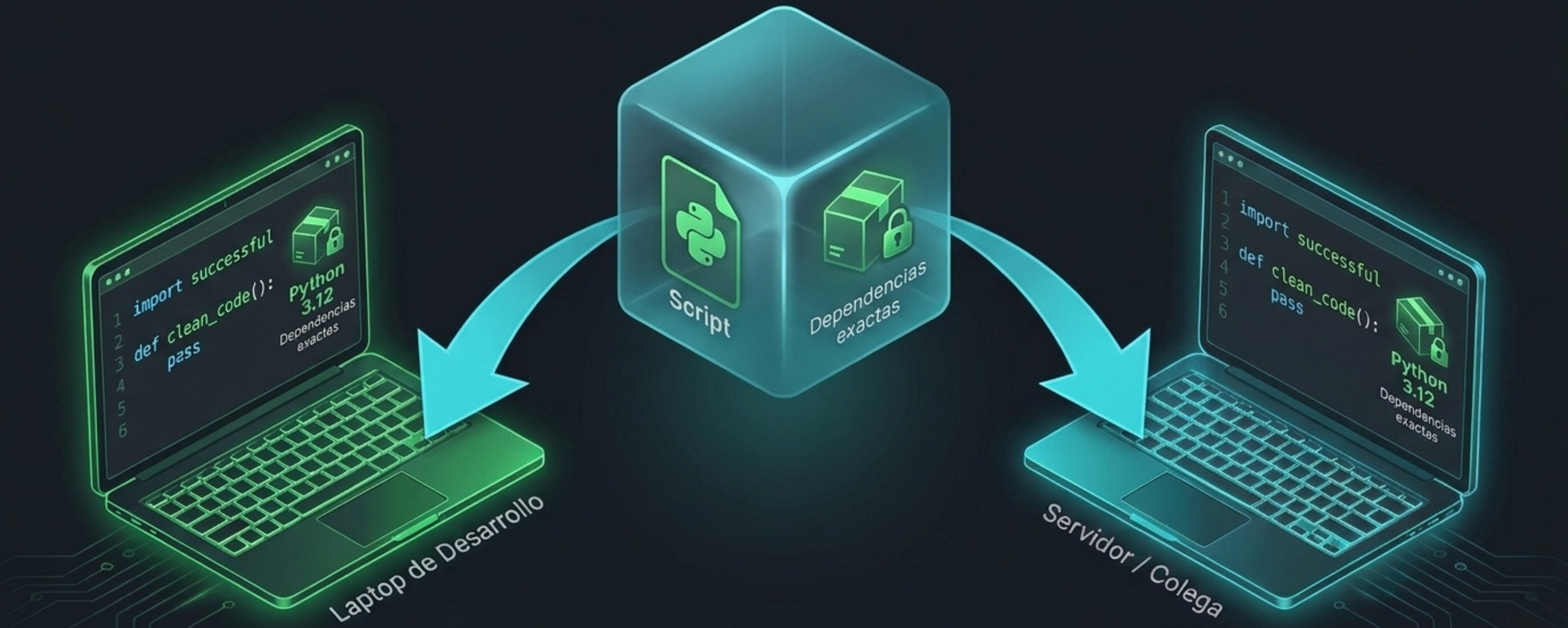


El código es idéntico, pero el entorno no lo es.

Conclusión: La consistencia del entorno es crítica para el despliegue exitoso.

Gemini Notebook

La solución es encapsular el entorno con el código.



Docker garantiza que el entorno de ejecución sea idéntico en cualquier lugar, eliminando las variables ocultas del sistema operativo anfitrión.

Gemini Notebook

Los cuatro pilares de la contenerización.

1. Dockerfile ------> **2. Imagen** ------> **3. Contenedor** ------> **4. Compose**



La receta paso a paso.



El paquete inmutable de solo lectura.



La instancia viva en ejecución.



El orquestador de múltiples servicios.

Gemini Notebook

Preparando la estación de trabajo cibernética.

Windows & macOS



Docker Desktop. Incluye el Engine, CLI y el plugin Compose listos para usar.

Linux



Docker Engine nativo + Plugin Compose V2. Recomendado para servidores nativos para evitar sobrecarga.

```
$ docker --version
Docker version 27.x.x, build xxxxx
$ docker compose version
Docker Compose version v2.x.x
```

Gemini Notebook

Práctica 1: Aislamiento de un script puro en Python.

main.py

```python
import sys

print(f'Hola desde Docker. Python {sys.version}')
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

Imagen base optimizada sin herramientas innecesarias.

Establece el directorio de trabajo.

Instalación aprovechando la caché de capas de Docker.

Ejecución del proceso principal.

Gemini Notebook

Construcción y ejecución de la primera imagen.



Gemini Notebook

El problema de la volatilidad de los datos.



Los contenedores son efímeros por diseño. Si un contenedor se detiene o se destruye, cualquier dato escrito en su capa interna desaparece para siempre. Necesitamos persistencia externa.

Gemini Notebook

Persistencia de datos: Volúmenes vs Montajes.

Montajes Directos (Bind Mounts)

Entorno de Desarrollo

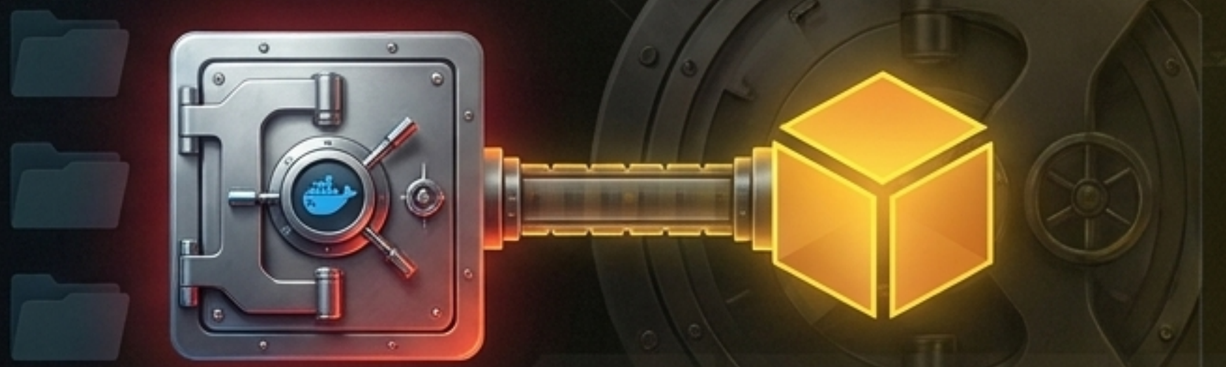


Uso: Sincronización de código fuente en tiempo real sin reconstruir la imagen.

```
- ./app:/app
```

Volúmenes Nombrados (Named Volumes)

Producción y Bases de Datos



Uso: Persistencia segura de datos (PostgreSQL/Redis) totalmente independiente del ciclo de vida del contenedor.

```
- pgdata:/var/lib/postgresql/data
```

Gemini Notebook

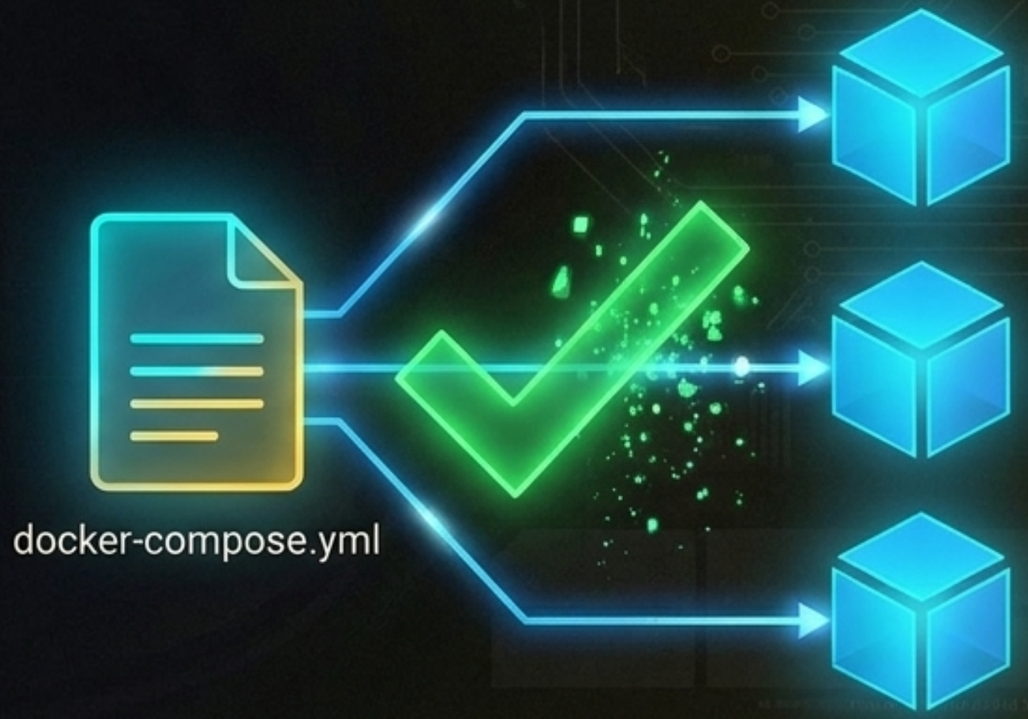
Orquestación de servicios múltiples con Compose

El Problema



⚠ Difícil de mantener y propenso a errores manuales.

La Solución

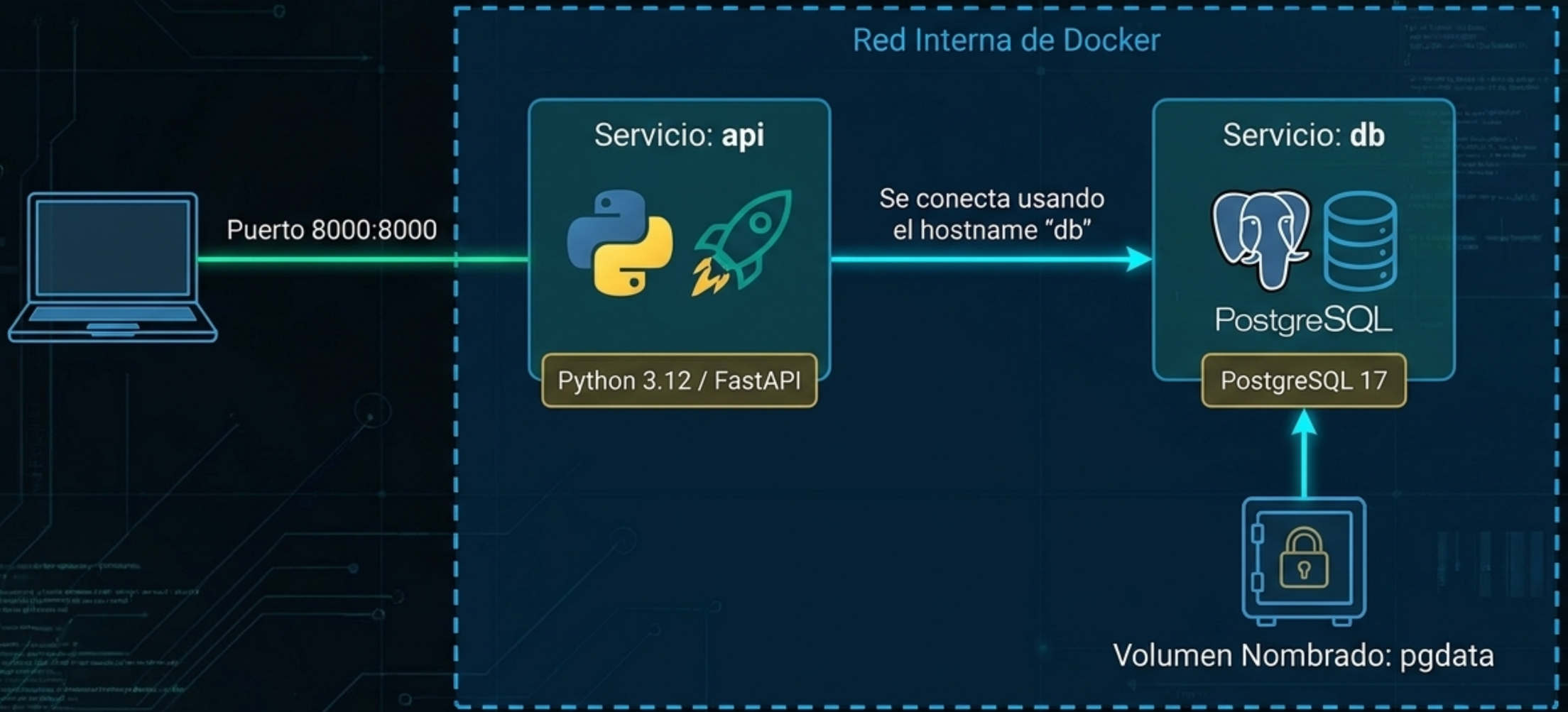


✓ Infraestructura como código: reproducible y declarativa.

Gemini Notebook

Premium Developer Workspace

Práctica 2: Arquitectura API REST local.



Gemini Notebook

Codificando el servicio de la API web.

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://app:secret@db:5432/app
```

Delega la construcción automática de la imagen al Dockerfile local.

Resolución mágica: El hostname de la base de datos es exactamente el nombre del otro servicio. No se necesitan direcciones IP fijas.

Gemini Notebook

Base de datos, persistencia y semáforos de salud.

Configuración de la Base de Datos

```
db:
  image: postgres:17
  volumes:
    - pgdata:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready"]
```



pgdata

depends_on:
condition:
service_healthy

Healthcheck
en progreso

¡Servicio listo!



El contenedor de la API espera pacientemente (Luz Roja).

Postgres arranca y ejecuta pruebas internas (Luz Amarilla).

La base de datos está lista. La API arranca sin fallar (Luz Verde).

Gemini Notebook

Despliegue y resolución mágica de red.

```
$ docker compose up --build
```



Network creada: práctica_2_default



Volume creado: práctica_2_pgdata



Container db: Iniciado



Container db: Healthy



Container api: Iniciado

Resolución de DNS interna

Docker Compose crea automáticamente una red privada. Los contenedores pueden comunicarse entre sí simplemente usando el nombre del servicio (ej. ping db).

Gemini Notebook

La infraestructura como código revoluciona el desarrollo.

El Pasado



El Presente (Docker)



Un entorno local limpio. Incorporación de nuevos desarrolladores en segundos. Despliegues totalmente predecibles. El código y su entorno son, **por fin, uno solo**.

Gemini Notebook